WSI ALM · CANDIDATE ASSIGNMENT · AI BUSINESS DISCOVERY TO POC

Groundwork

A tool that turns a client's scattered messages, calls and documents into a plan you can build from — and can prove where every sentence of that plan came from.

WHAT IT IS	A working web application, built end to end
BUILT BY	Ashika
TIME TAKEN	One week
SIZE	~130 files · 129 automated tests · 10 browser tests
STATUS	Complete and running locally; ready to deploy
THIS DOCUMENT	Explains the whole thing in plain language

Part 1

What the company asked for

The assignment describes a problem that consultants live with every day, and asks you to solve it with software.

Their brief said this. When a client hires you, their requirements never arrive as one clean document. They arrive as two recorded meetings, a WhatsApp group full of messages and photos, a PDF someone wrote two years ago, screenshots of the spreadsheet they currently use, and a website.

A consultant reads all of it, works out what the client actually needs, spots what is going wrong today, suggests a better way of working, and then builds a small demo to show what the fix could look like.

WSI ALM asked for an application that does that. Their five requirements were:

1. **Take client inputs.** At least three different kinds — transcripts, chat exports, PDFs, documents, screenshots, a website.
2. **Understand the business need.** The main goal, how things work today, what hurts, what is required, and what nobody has told you yet.
3. **Suggest a better process.** Practical and easy to understand.
4. **Create a solution outline.** Features, user roles, screens, and how it all flows together.
5. **Create a basic POC.** A small working demo of the idea.

They also said what they would judge: how well you understand the business problem, how practical your solution is, how you use AI, how the frontend and backend work together, your engineering quality, and how clearly you can explain your work.

The second, hidden assignment

Alongside the brief came a job description, and it asks for something quite different. It wants someone who can build *multi-tenant SaaS* — software where many separate customers use the same application, and none of them can ever see another's data. It lists strict data isolation, login and permissions, database design, testing, and continuous integration.

And under a heading called **Not A Fit If**, it says: "*You rely on AI without understanding the output*."

THE DECISION THAT SHAPED EVERYTHING

Most people answering this brief would build an upload box connected to an AI. That answers the first assignment and completely ignores the second.

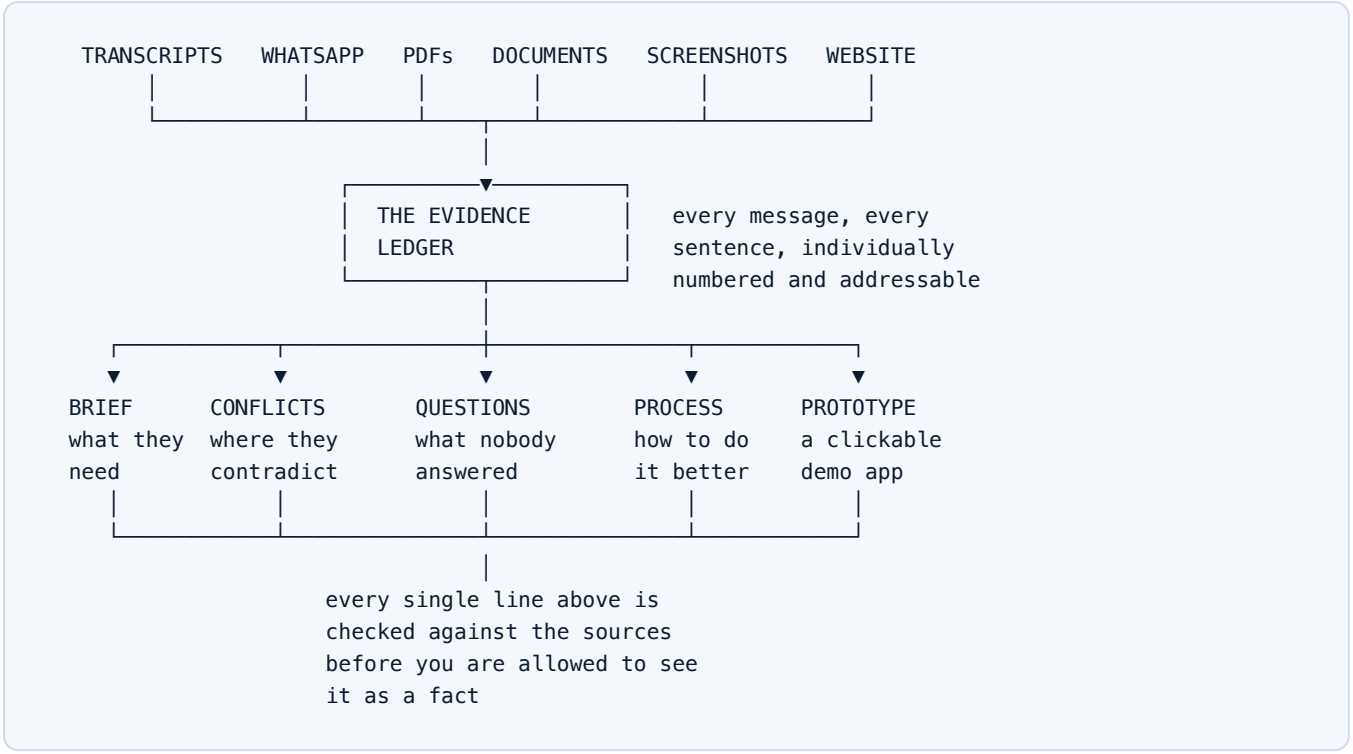
So Groundwork is built as a **real multi-tenant product** — with organisations, user roles, and database-enforced separation between customers. And its single most important feature is a mechanism that **refuses to trust the AI's output** without checking it first. That is a direct answer to the line they used to screen people out.

Part 2

What Groundwork does

In one sentence: you give it the mess, and it gives you a plan where every claim can be traced back to the exact sentence somebody actually said.

Here is the whole thing as a picture.



The five stages below match the company's five requirements exactly, one for one.

1

Take the inputs

You drag in files, or paste a website address. It works out what each file is by looking inside it, not by trusting the file name — because people rename things, and a WhatsApp export usually arrives called `_chat.txt`.

Handles: Teams and Zoom transcripts · WhatsApp exports · PDFs · Word documents · screenshots · websites

2

Understand what they need

It reads everything and writes a discovery brief: the real goal, who the people are and what each one cares about, how the work happens today step by step, where it hurts, what is required, and what the client has explicitly ruled out.

Also produces: contradictions between sources · the questions nobody answered

3

Suggest a better way

Two diagrams side by side — how it works now, and how it could work. Every proposed change says exactly what waste it removes, in the client's own words.

`Example: "removes the day-long round trip that starts every time a client rings the founder"`

4

Outline the solution

Who the users are and what each can do, what screens exist, and a feature list sorted into Must have, Should have, Could have, and Won't do yet.

`Every feature shows the evidence behind it — or admits it was the tool's own idea`

5

Build a demo they can actually test

Not a picture of an app — a small working tool. Post a progress update and it appears on the client's own screen. Post something involving a date or a cost and it queues for approval instead, reaching the client only once approved. That is the proposed way of working, and the client can try it and decide whether it holds up.

`Shareable with a link the client can open without an account`

Part 3

The one idea that matters most

AI tools make things up. This is the well-known problem with them, and it is the reason the job description says "not a fit if you rely on AI without understanding the output."

Groundwork's answer is a rule the software enforces on itself:

THE RULE

The AI is not allowed to state anything unless it also provides the exact sentence, copied word for word, from the document it came from. The software then goes and checks that the sentence really is in that document. If it is not there, the claim is shown to you in red as unsupported.

Why this is unusual

Most AI tools give you an answer and you either trust it or you don't. There is no way to check without re-reading everything yourself, which defeats the point of the tool.

> *Think of it like a school essay where every sentence must have a footnote — and a teacher who actually opens each book to confirm the quote is real before marking it.*

In Groundwork, every claim carries a small tag showing which source it came from. Click the tag and a panel slides open showing that document with the exact sentence highlighted in place. You can check any statement in about two seconds.

The score, and why it is not 100%

At the top of the brief is a number. In the demo it reads:

84% — 27 of 32 claims verified against source

5 could not be traced to a quote — including 4 marked as assumptions.

A perfect score would actually be suspicious. Here is what the missing 17% is made of:

- **Four are assumptions the AI declared itself.** Things it had to guess because nobody said them — like "we assume the accounting system has a way to read payment status." It flags these openly instead of hiding them inside confident-sounding sentences.

- **One is a deliberate trap.** I planted a fake requirement that cites a sentence appearing in none of the documents. It shows up in red with a cross, and the panel says the quote could not be located.

> WHY THE TRAP IS THERE
>
> A safety check that has never been seen failing is not proof of anything. The planted claim means anyone can watch the mechanism catch a lie — and the automated tests check both halves: that all the real citations pass, *and* that the fake one still fails. If someone accidentally breaks the checking, the tests go red immediately.

Being forgiving without being fooled

The check has to be sensible. AI models often change a curly apostrophe to a straight one while copying, or reflow a sentence that was split across two lines. That is still an honest copy, and rejecting it would flag good work as fake.

HOW IT MATCHED	WHAT THAT MEANS	COUNTED AS VERIFIED?
Exact	Character for character identical	YES
Normalised	Same words; punctuation or spacing differs	YES
Approximate	Nearly the same sentence, a word or two off	NO — SHOWN AS APPROXIMATE
Not found	This sentence is not in that document	NO — SHOWN AS UNSUPPORTED

Forgiving about punctuation, unforgiving about wording. A model that *reworded* a sentence has not quoted it, so that never counts as verified.

Part 4

Walking through it, as a user would

The demo uses a realistic invented client: Nova Interiors, a 40-person interior design firm in Pune that runs every project through WhatsApp groups and one shared spreadsheet.

Their inputs are two recorded consultant calls, a WhatsApp group with 69 real-looking messages, their internal process document as a Word file, a vendor terms PDF, and a screenshot of their tracking spreadsheet.

Hidden inside that material, on purpose, are the kinds of problems a real engagement has: a budget stated as two lakh by the founder and five lakh by the operations head two weeks later; an argument about who is allowed to approve messages to clients; a whole module that only appears in the second call; and several important questions nobody thought to ask.

What happens when you press the button

You press "Run discovery." Six stages appear one after another as they finish, so you watch the work arrive rather than staring at a spinner:

```
✓ brief      27 of 32 claims verified against source (84%)
✓ conflicts  3 contradictions found
✓ questions  7 open questions
✓ process    5 changes proposed
✓ outline    9 features, 5 must-have
✓ prototype 4 screens
```

The three moments worth showing anyone

One. Click any claim and see the proof. The brief says the founder is acting as a switchboard between clients and site teams. Click the tag beside it and you see the transcript, with his actual words highlighted: "*If I stop being the switchboard.*"

Two. The contradiction, side by side. The Conflicts screen shows the budget disagreement with both quotes next to each other — the founder saying two lakh on the 12th, the operations head reporting five lakh on the 26th. You choose which one stands, add a note, and it is recorded as a decision.

Three. Questions you can actually send. The tool lists what nobody answered, then composes them into a message you can paste straight into WhatsApp or email. Not a list of gaps — a message ready to send.

THE THING THAT SURPRISES PEOPLE

Decisions and answers are not just stored — they are fed back in. When you run the analysis a second time, the tool already knows the budget was settled at five lakh and stops raising it as a problem. The engagement **accumulates knowledge** instead of starting from zero every time.

And what the client sees

You create a link and send it. The client opens it with no account, no app, no password — and can forward it to their spouse or their accountant, which is exactly what this fictional client asked for.

They see the goal, the problems, the proposed new process and the demo. They deliberately do **not** see the contradictions (which quote their own staff disagreeing with each other) or the assumptions list. Those are your working notes, not theirs.

Part 5

Why there is one repository, not two

You asked about this specifically, and it is a fair question — many companies do split frontend and backend into separate codebases.

First, what those two words mean

FRONTEND

Everything you see and click. Buttons, pages, the panel that slides open when you click a citation. It runs in your web browser.

BACKEND

Everything you don't see. Reading files, talking to the database, checking passwords, calling the AI, verifying quotes. It runs on a server.

Groundwork has both. It has a full backend: a database with twelve tables, login and permissions, file processing, six AI calls, and the verification engine. And it has a full frontend: a dozen screens with live updates, sliding panels, diagrams and forms.

They live in one repository because the framework this is built on — Next.js — is designed to hold both. This is not a shortcut. It is how the company itself builds software.

THE DIRECT ANSWER

The WSI ALM job description asks for "**a multi-tenant SaaS architecture using Next.js, Vercel and Google Cloud.**" Next.js is a framework where the frontend and backend live together by design, and Vercel is the company that makes Next.js. Splitting them into two repositories would mean working against the exact tool they named.

How they are separated inside the one repository

One repository does not mean one jumbled pile. The code is clearly divided by what each part is responsible for:

FOLDER	WHAT LIVES THERE	RUNS WHERE
`src/app/api/`	The backend API. Login, file upload, generating the analysis, sharing links.	Server
`src/db/`	Database structure and the rules that keep customers separated.	Server
`src/lib/`	The engine room: file readers, the quote verifier, the AI layer.	Server
`src/components/`	The interface — buttons, panels, diagrams.	Browser
`src/app/**/page.tsx`	The screens themselves.	Both

The backend still exposes a proper API that anything could call. You can talk to it with plain web requests, and I tested it that way throughout — logging in, uploading a PDF, running the analysis, all without opening a browser.

What splitting them would have cost

- **Two deployments instead of one.** Two things to set up, two to keep in sync, two to break.
- **Duplicate type definitions.** The shape of a "project" or a "citation" would have to be written twice and manually kept identical. Here it is written once, and the computer catches any mismatch instantly.
- **Slower pages.** Screens can currently ask the database directly while rendering. Split apart, every screen would need an extra network round trip.
- **Working against the framework and against the job spec.**

A split makes sense when separate teams own each half, or when several different applications share one backend. Neither is true here — and if it became true later, the API layer is already cleanly separated and could be lifted out.

Part 6

How it is built, explained simply

This section covers the technical decisions and why each one was made. Every term is explained the first time it appears.

Keeping customers apart — the most important engineering problem

Multi-tenant means many companies use the same application. Meridian Digital and Northwind Studio both have accounts. Neither must ever see the other's work — not by accident, not by guessing a web address, not because a developer forgot something.

Groundwork does this **twice, in two independent places**:

LAYER 1 — IN THE CODE

Every request for data says which company is asking. Every database question includes "…and only for this company."

LAYER 2 — IN THE DATABASE

The database itself has rules built in. Even if the code forgets to ask correctly, the database returns nothing rather than someone else's data.

> *Layer 1 is a receptionist who checks your badge. Layer 2 is a lock on every door that only opens for that badge. You want both, because receptionists have off days.*

A REAL BUG THIS CAUGHT, WORTH TELLING YOU ABOUT

When I first wrote the database rules, I also wrote tests to prove they worked. **All five tests failed.** Every company could see every other company's data.

The reason is a genuine trap in this technology: database rules are skipped for the account that *owns* the tables — which is exactly the account applications normally connect as. The rules existed, looked correct, and did nothing.

The fix is that the application now drops to a restricted, lower-privilege identity before touching any data. This is the kind of mistake that ships to production looking perfectly fine. It is now covered by tests that would fail loudly if anyone undid it.

The database, and why setup takes two commands

Normally, running someone else's project means installing a database server first — a fiddly step that stops many reviewers before they start.

Groundwork uses **PGlite**: a real PostgreSQL database compiled to run inside the application itself, storing everything in a local folder. No server to install. `npm install`, then `npm run setup`, then it runs.

For real deployment it switches to a hosted database (Neon) by setting one configuration value. Same structure, same rules, same everything — only the connection changes.

Login and permissions

When you sign in, the server gives your browser a *signed token* — like a wristband at a venue that cannot be forged because it carries a cryptographic seal only the server can produce.

There are three roles: **owner**, **consultant** and **client**. A client is bounced out of the consultant screens automatically.

I wrote this by hand rather than installing a login library, because the job description asks specifically for understanding of tokens and permissions. A library would have hidden exactly the thing they wanted to see. It is around a hundred readable lines.

How the AI part works

Six separate calls to Claude, one per stage, rather than one giant call. This matters for three reasons: each one finishes comfortably within time limits, a failure at the last stage does not destroy the work already done, and you can watch progress arrive.

All six calls read the same documents. Sending those documents six times would be slow and expensive, so the system uses **prompt caching** — the documents are sent once and remembered, and the later five calls reuse them at roughly a tenth of the cost.

A DECISION I MADE DELIBERATELY, AND WHY

There is a popular technique called RAG, where documents are chopped into fragments and only the "relevant" pieces are fetched for each question. It is the fashionable answer.

I did not use it, on purpose. These documents total roughly 50,000 to 200,000 words — small enough to give the AI all of it at once. RAG would have added a lot of machinery in order to make the result *worse*, because understanding a client means noticing that something said in a call contradicts a WhatsApp message three weeks later. Chopping documents into fragments is precisely how you lose that.

Knowing when *not* to use a fashionable technique is itself an engineering judgement, so it is documented rather than silently omitted.

Screenshots

A screenshot is turned into text the moment it is uploaded, not when the analysis runs. That ordering is deliberate: it means anything the tool says about a screenshot can be quoted and verified just like a transcript. Feeding images straight into the analysis would put that content outside the checking system — the one place this product cannot afford a blind spot.

Testing

KIND	COUNT	WHAT IT PROTECTS
Automated tests	100	File readers, the quote verifier, customer separation, share links, decisions surviving a re-run
Browser tests	9	The actual demo, clicked through automatically start to finish

The browser tests are the demo itself, performed by a robot. They sign in, run the analysis, click a citation, check the highlighted sentence appears, confirm the planted fake claim still shows as unsupported, and confirm the other company still gets a "not found" page. If any part of the story breaks, it is caught automatically rather than discovered live in front of someone.

Two security decisions worth naming

The generated demo runs in a locked box. The prototype is code an AI wrote, so it is displayed inside a sealed frame with no access to the main application, its data, or the network. One browser test checks the exact security setting, so nobody can weaken it later to silence a warning.

The website reader cannot be turned against us. Letting users give the server a web address to fetch is a classic security hole — someone could point it at internal systems. Private and internal addresses are refused, and the address is looked up and re-checked before anything is fetched.

Part 7

How the work was organised

Each feature was built on its own branch and merged separately, so the history reads as a sequence of decisions rather than one enormous dump of code.

BRANCH	WHAT IT ADDED
`feat/share-links`	Client links that work without an account and can be revoked
`feat/conflict-resolution`	Deciding a contradiction, and feeding that decision into the next run
`feat/website-ingestion`	Reading a client's website, with protection against misuse
`feat/screenshot-vision`	Turning screenshots into quotable evidence
`test/e2e-playwright`	Robot browser tests over the whole demo
`feat/question-workflow`	Working a question through to a recorded answer
`feat/artifact-versions`	Seeing every previous run and comparing their scores
`chore/deployment-config`	Deployment settings and security headers
`docs/submission`	Documentation brought up to date

Every commit message explains the reasoning, including what broke. Three genuine bugs were found and fixed this way:

- **Re-running destroyed your decisions.** Running the analysis a second time deleted every contradiction — including ones already decided. Hours of a consultant's work, gone silently.
- **The same bug in a different shape** in the questions list, which wiped and recreated every open question on each run.
- **Two test systems fighting.** Each passed alone while the combined run failed, which would have turned the build red on the first push.

Part 9

Everything else it does

Four things built after the five stages were working, each closing a gap that showed up once the thing was usable.

A team, not one person

A workspace is shared. An owner invites someone by email and role; they open a link, choose a password, and are in. Everyone then works on the same projects and adds to the same pile of evidence.

Invitations expire after a week, work only once, and can be revoked. The email address is fixed by the invitation and cannot be edited by whoever holds the link — otherwise anyone who got hold of one could sign up as somebody else. And the last owner cannot be removed, because that would leave the workspace with nobody able to manage it and no way back in.

Evidence can be taken away, not just added

Documents can be withdrawn. Everything derived from one goes with it — otherwise a quotation would be left pointing at a document that no longer exists, and would show up in red as unsupported. That would look like the AI had invented something when in fact a consultant simply removed a file, which is the worst possible way for this particular product to be wrong.

Existing briefs are left untouched. A brief is a record of what was concluded from the evidence available at the time; re-running the analysis is what produces a new version without the withdrawn document.

Text can also be typed or pasted straight in. The most common thing a client sends is not a file but a paragraph in an email, and making somebody save that to a text file first is a pointless step.

Comparing two runs

Every analysis is kept. The Compare screen puts two of them side by side and says what appeared, what dropped away, and what stayed but was reworded — plus whether the grounding score went up or down.

The tricky part is that the AI reorders things between runs and rewords things it still means. If a line that simply moved were reported as "dropped, and a new one added", the comparison would be full of false alarms and nobody would read it twice. So lines are matched on what they say, not where they sit, and a near-match is shown as a rewording with the old wording struck through beneath it.

A document the client can keep

The client can download the proposal. It is a Word-compatible text document rather than a PDF, deliberately: it opens anywhere, it can be pasted into an existing proposal template, and it survives being edited by the consultant before it goes out — which is what actually happens to these documents.

It carries what was understood, what is not working, what is proposed and what would be built — plus the honesty score and the full list of assumptions, so the client can see how much of it rests on guesswork and correct anything wrong. It deliberately leaves out the contradictions: quoting a client's own staff disagreeing with each other is a working note for the consultant, not something to hand the client.

One screen, in the order the work happens

Signing in used to land on a list of projects, and opening one gave eight equal tabs with no indication of which to look at first — on a project that had nothing in it yet. Everything was present and the order was left to the reader to work out.

The dashboard now puts the three steps in sequence: add what the client gave you, create the plan, read it. The third does not appear until there is a plan to read. The deeper pages are unchanged and still hold the real work; they are now somewhere you go *from* a plan rather than the first thing you meet.

Anyone can create their own workspace, and it starts genuinely empty — no project, no documents, nothing copied across. The demo material lives in one account and only there. That matters more than it sounds: if another firm's engagement were sitting in your workspace, you could not tell what the tool had read from your own documents and what was already there.

Part 10

What is done, and what is not

Being straight about the edges is part of the work. Here is the honest position.

Built and working

DONE	All five stages the assignment asked for
DONE	Seven input types, detected from file contents; sources can be withdrawn too
DONE	The verification system and its visible score
DONE	Contradiction detection, decisions, and the questions workflow
DONE	Multi-tenant separation, enforced twice and tested
DONE	Sign-up and login with three roles; a new workspace starts empty
DONE	Client share links, revocable · teams and invitations · downloadable proposal
DONE	Version history, with a side-by-side comparison of any two runs
DONE	184 automated tests, 12 browser tests, continuous integration

Not done, deliberately

PENDING	**Deployment.** All configuration is written; it needs hosting accounts, which is your step.
CHOICE	**A timeline of how requirements changed.** Useful, but the five required stages came first.
CHOICE	**Chat with the evidence.** Same reasoning.
CHOICE	**Sending invitations by email.** Today the owner copies the link across themselves.

Those three were on the plan from day one as optional, and cutting them was the plan working rather than the plan failing. A tight, complete, well-tested story is worth more than a wider one with thin patches — and saying so plainly is itself part of the answer to "how clearly can you explain your work."

TO RUN IT YOURSELF

```
npm install
cp .env.example .env
npm run setup
npm run dev
```

Then sign in as `ashika@meridian.example` with the password `demo1234`. No database to install, no accounts to create. It works without an AI key too — it replays a recorded analysis and says so clearly, and the quote checking still runs for real against the real documents.

Groundwork · built for the WSI ALM candidate assignment · August 2026